TP 2 - Programmer avec Unity

L'objectif du TP est d'enrichir la scène de bowling en ajoutant des scripts pour programmer la logique du jeu. Pensez à publier régulièrement votre progression sur GitHub.

- Mise en place
  - Ouvrir Unity Hub et rouvrir le projet `EDN-XR-Project`
  - Double-cliquer sur la scène `Bowling VR` dans le panneau Projet pour l'ouvrir

Introduction au scripting dans Unity

Unity utilise le C# comme langage de programmation. C# est un langage orienté objet : le code est organisé en classes qui encapsulent les champs et les méthodes.

Dans Unity, chaque script est une classe qui représente un composant pouvant être attaché à un GameObject. Le composant définis par le script permet de définir le comportement à suivre du GameObject. Chaque script hérite de la classe `MonoBehaviour` afin d'être interprété comme un composant par Unity et peut implémenter des méthodes *callbacks* appelées automatiquement à des moments précis.

Au sein d'un `MonoBehaviour`, vous pouvez accéder aux propriétés du GameObject auquel il est attaché. Par exemple, `transform.position` retourne les coordonnées du GameObject dans la scène.
Les méthodes les plus courantes sont :

- Start : appelée une seule fois au démarrage de la scène pour initialiser l'objet
- Update : appelée à chaque pas de temps pour mettre à jour l'état de l'objet

Pour créer un script, faire `clic droit` dans le panneau Projet sur le dossier `Bowling` puis `Create > C# Script`. Unity va générer un fichier .cs avec la structure de base (classe qui hérite de `MonoBehaviour`...)

> **Note** : Le nom du fichier et le nom de la classe à l'intérieur du fichier doivent être identiques. Si vous renommez un fichier après sa création, pensez à renommer la classe en conséquence dans le code.

Implémentation de fonctionnalités

Chaque fonctionnalité impliquera un ou plusieurs scripts. Exécuter la scène dans le casque pour valider le fonctionnement des scripts (reprendre les instTP 1 pour lancer l'application dans le casque)

Détection des quilles tombées

Le premier script sera attaché à chaque quille et permettra de savoir si celle-ci est tombée. Pour déterminer si la quille est tombée, on utilise une règle simple : si l'angle d'inclinaison de la quille dépasse un seuil, elle est considérée comme tombée.

- Créer le script
  - Créer un nouveau script C# nommé `FallingPin` dans le dossier `Bowling`
  - Double-cliquer sur le script pour l'ouvrir dans votre éditeur de code
- Ajouter deux champs publics à la classe
  - `float fallAngleThreshold` : le seuil d'inclinaison en degrés à partir duquel la quille est considérée comme tombée (fixer la valeur par défaut à 45)
  - `bool isFallen` : indique si la quille est tombée (initialiser à `false`, la quille debout par défaut)
- Implémenter dans `Update` la logique de détection
  - Calculer l'angle entre l'orientation locale de la quille (`transform.up`) et le haut du monde (`Vector3.up`) en utilisant la méthode Vector3.Angle
  - Si l'angle d'inclinaison est inférieur à `fallAngleThreshold`, passer `isFallen` à `true`
  - Puisqu'une quille tombée ne se remet pas debout d'elle-même, cette vérification ne doit être effectuée que si `isFallen` est à `false`
- Attacher le script aux quilles
  - Glisser-déposer le script `FallingPin` dans le panneau projet sur tous les GameObjects `Pin` dans la hiérarchie
  - En cliquant sur un `Pin`, le composant script `FallingPin` est visible dans l'inspecteur
  - Les champs publics d'un script sont aussi visibles dans l'inspecteur du composant

> **Note** : Pour vérifier que la détection fonctionne, vous pouvez faire disparaître une quille lorsqu'elle est tombée. Pour cela, ajouter au début de Update l'instruction gameObject.SetActive(!isFallen) pour désactiver de la scène le GameObject associé dès qu'il est détecté comme tombé.

Comptage des quilles tombées

Nous allons créer un script afin de comptabiliser le nombre de quilles tombées. Pour cela, le script va récupérer tous les objets dans la scène étant de type FallingPin, et compter ceux marqués comme tombés.

- Créer le script de comptage
  - Créer un nouveau script C# PinCounter dans le dossier Bowling
  - Associer le script au GameObject BowlingAlley dans la hiérarchie
- Implémentation du compteur
  - Ajouter un champ FallingPin[] pins pour stocker la liste des quilles
  - Ajouter un champs int fallenCount pour compter le nombre de quilles tombées (initialisé à 0)
  - Utiliser GetComponentsInChildren pour récupérer tous les composants FallingPin dans les enfants de BowlingAlley et les stocker dans pins
  - Parcourir le tableau pins dans la méthode Update et compter dans fallenCount le nombre de quille où isFallen est à true

> **Note** : GetComponentsInChildren est une opération coûteuse car elle parcourt toute la hiérarchie enfant à chaque appel. Il vaut mieux l'appeler une seule fois dans Start pour initialiser le tableau pins, et réutiliser ce tableau dans Update plutôt que de refaire la recherche à chaque pas de temps.

Affichage du score

- Créer une UI dans la scène
  - Dans la hiérarchie, créer un Canvas (clic droit > UI > Canvas) et le renommer ScoreUI
  - Dans l'inspecteur, changer le Render Mode en World Space, ajuster sa position pour qu'il soit visible dans la scène (par exemple au-dessus de la piste), mettre Width à 1, Height à 1 et Scale à 1, 1, 1
  - Ajouter un élément texte ScoreText (clic droit > UI > Text - TextMeshPro) en enfant de ScoreUI
  - Dans l'inspecteur, mettre Width à 10, Height à 10, Scale à 0.1, 0.1, 0.1 et Font Size à 5
- Modifier le script PinCounter pour afficher le score sur l'UI
  - Déclarer un champ public TextMeshProUGUI scoreText pour créer une référence vers le texte de l'UI
  - Dans la méthode Update, après le calcul du score, mettre à jour le champs scoreText.text (le nombre de quille tombées doit être convertit en chaine de caractères)
  - Dans l'inspecteur Unity, sélectionner le GameObject BowlingAlley et glisser-déposer ScoreText dans le champ ScoreText du composant PinCounter

Script de réinitialisation

Nous allons mettre en place un dernier script pour remettre la boule et les quilles à leur position d'origine. Ce script va recharger simplement la scène quand l'utilisateur appui sur un bouton du contrôleur.

- Créer le script
  - Créer un nouveau script C# BowlingResetter dans le dossier Bowling
  - Associer le script au GameObject BowlingAlley dans la hiérarchie
- Implémenter le rechargement dans Update
  - Déclarer un champ public InputActionReference resetAction pour le bouton de réinitialisation (dans l'inspecteur du composant vous pouvez fixer ce bouton à XRI RightHand Interaction/Activate)
  - Dans la méthode Update, utiliser la propriété resetAction.action.triggered pour détecter l'appui sur le bouton.
  - En cas d'activation, utiliser la méthode SceneManager.LoadScene en lui passant comme paramètre le nom de la scène actuelle pour recharger la scène (le nom de scène courante peut être récupérer avec SceneManager.GetActiveScene().name)

Pour aller plus loin

Au-delà des règles de jeu, le scripting sert à définir les réponses de l'environnement. Ces composants permettent de lier les interactions physiques à des effets sonores et visuels personnalisés afin de rendre la scène plus immersive.

Audio de collision

Unity permet de déclencher un son lors d'un contact physique entre deux objets via la méthode `OnCollisionEnter`.

- Importer un fichier audio (`.wav` ou `.mp3`) dans le dossier `Bowling`
- Sur chaque `Pin`, ajouter un composant `Audio Source` et lui assigner le fichier audio dans le champ `Audio Clip`
- Créer un script `PinAudio`
    - Le script doit implémenter la méthode `OnCollisionEnter` qui est appelée quand le Gameobject est en collision
    - Pour jouer un son, il suffit de récupérer le composant `AudioSource` avec la méthode `GetComponent` et appeler la méthode `Play`

Effets de particules

Un *Particle System* permet de créer des effets visuels dynamiques, comme des confettis, lorsqu'une quille tombe.

- Créer l'effet
    - Dans la hiérarchie, créer un Particle System (`clic droit > Effects > Particle System`), nommer-le `CelebrationEffect` et faites en un enfant de `BowlingAlley`
    - Positionner l'effet à un endroit stratégique (par exemple, au-dessus des quilles ou en fond de piste)
    - Dans l'inspecteur, décochez l'option Play on Awake*
    - Explorer ses propriétés dans l'inspecteur (taux d'émission, durée de vie, couleur, gravité) pour obtenir l'effet voulu
- Dans le script `FallingPin`
    - Déclarer un champ public `ParticleSystem celebrationEffect` pour recevoir la référence de l'effet
    - Dans `Update`, appeler la méthode `Play` au moment où `isFallen` passe à `true`
    - Dans l'inspecteur, sélectionner toutes les quilles dans la hiérarchie et glisser-déposer le Gameobject `CelebrationEffect` dans le champs `Celebration Effect` de `FallingPin`